EXPERIMENT 1

Job Scheduling and Automation in Linux – Step-by-Step Practical Code

Aim: To create a Bash script, execute it manually, schedule it using Cron, and verify automated execution through log files.

What you will build: A simple backup script that copies the Documents folder into a backup folder and records its execution in a log file.

PART A – CRON JOB SCHEDULING (MAIN PRACTICAL)

Step 1: Create a folder for scripts and logs

```
mkdir -p ~/scripts
mkdir -p ~/logs
cd ~/scripts
```

Explanation: `mkdir -p` creates the directories if they do not already exist. `cd` changes into the scripts directory.

Step 2: Create the backup script

```
nano backup.sh
```

Type the following code inside Nano:

```
#!/bin/bash

echo "----------------------------"
echo "Backup Started: $(date)"
echo "----------------------------"

mkdir -p "$HOME/backup"

if [ -d "$HOME/Documents" ]; then
    cp -r "$HOME/Documents" "$HOME/backup/"
    echo "Backup Completed Successfully"
else
    echo "Documents folder not found"
fi
```

Save: Press Ctrl + O, press Enter, then Ctrl + X.

Explanation: The script uses Bash, prints the date and time, creates a backup directory, checks whether Documents exists, copies it recursively, and displays the result.

Step 3: Give execution permission

```
chmod +x backup.sh
```

Explanation: Makes the script executable.

Step 4: Run the script manually

```
./backup.sh
```

Expected result: A message showing the backup start time followed by “Backup Completed Successfully” (if the Documents folder exists).

Step 5: Verify the backup folder

```
ls ~/backup
```

Explanation: Lists the contents of the backup directory and lets you check whether the Documents folder was copied.

Step 6: Open the Cron editor

```
crontab -e
```

If prompted to select an editor, choose Nano. Move to the bottom of the file using the down-arrow key. Do not delete the explanatory comment lines beginning with #.

Step 7: Schedule the script every two minutes

Add this as a new line at the bottom of the crontab file:

```
*/2 * * * * /bin/bash /home/USERNAME/scripts/backup.sh
```

Important: Replace `USERNAME` with your Linux username. Find it using `whoami`. You can also use the full path printed by `echo $HOME`.

Save the Cron file using Ctrl + O → Enter → Ctrl + X.

Step 8: Verify the scheduled Cron job

```
crontab -l
```

Expected result: The Cron line you added should be displayed.

Step 9: Check Cron service status

```
systemctl status cron
```

If Cron is inactive and you have permission, start it with:

```
sudo systemctl start cron
sudo systemctl enable cron
```

On some Linux distributions the service may be named `crond` instead of `cron`.

Step 10: Check the log after Cron runs

To store the script's output in a log file, edit the Cron entry again:

```
crontab -e
```

Replace the earlier entry with:

```
*/2 * * * * /bin/bash /home/USERNAME/scripts/backup.sh >> /home/USERNAME/logs/backup.log 2>&1
```

Wait at least two minutes, then check the log:

```
cat ~/logs/backup.log
tail -f ~/logs/backup.log
```

Explanation: `>>` appends output to the log instead of overwriting it. `2>&1` sends error messages to the same log. Press Ctrl + C to stop `tail -f`.

PART B – IMPORTANT CRON COMMANDS

Cron expression	Meaning
* * * * *	Every minute
*/2 * * * *	Every 2 minutes
*/5 * * * *	Every 5 minutes
*/10 * * * *	Every 10 minutes
0 * * * *	At the start of every hour
0 9 * * *	Every day at 9:00 AM

Cron field order: minute hour day-of-month month day-of-week command

```
* * * * * command
| | | | |
| | | | +-- Day of week (0-7)
| | | +---- Month (1-12)
| | +------ Day of month (1-31)
| +-------- Hour (0-23)
+---------- Minute (0-59)
```

Useful commands

Command	Purpose
crontab -e	Edit scheduled Cron jobs
crontab -l	List scheduled Cron jobs
crontab -r	Remove all of the current user's Cron jobs – use carefully
whoami	Display the current username
echo $HOME	Display the user's home directory
chmod +x file.sh	Make a script executable
./file.sh	Run a script from the current directory
cat file.log	Display the contents of a log file
tail -f file.log	Continuously monitor new log entries
systemctl status cron	Check the Cron service status

PART C – SYSTEMD TIMERS (ADDITIONAL TOPIC)

A systemd timer is another Linux scheduling method. It activates a systemd service at a specified time or interval. This section is additional; the Cron procedure above is the main practical workflow.

Step 1: Create a service unit

```
sudo nano /etc/systemd/system/backup.service
```

Enter:

```
[Unit]
Description=Backup Service

[Service]
Type=oneshot
ExecStart=/bin/bash /home/USERNAME/scripts/backup.sh
```

Step 2: Create a timer unit

```
sudo nano /etc/systemd/system/backup.timer
```

Enter:

```
[Unit]
Description=Run Backup Periodically

[Timer]
OnBootSec=2min
OnUnitActiveSec=2min
Unit=backup.service

[Install]
WantedBy=timers.target
```

Replace USERNAME with your Linux username in the service file. Save each file using Ctrl + O → Enter → Ctrl + X.

Step 3: Reload systemd and enable the timer

```
sudo systemctl daemon-reload
sudo systemctl enable --now backup.timer
```

Step 4: Check the timer and service

```
systemctl list-timers
systemctl status backup.timer
systemctl status backup.service
```

Useful systemd commands

```
sudo systemctl start backup.timer
sudo systemctl stop backup.timer
sudo systemctl restart backup.timer
sudo systemctl disable backup.timer
journalctl -u backup.service
```

Note: Creating system-wide unit files normally requires sudo privileges. If you do not have administrator access in the lab, learn the commands and explain the concept rather than attempting to create these files.

PART D – TROUBLESHOOTING AND EXAM REVISION

Common problems and solutions

Problem	What to check / solution
Permission denied	Run chmod +x script.sh; check permissions with ls -l.
File not found	Check the filename and use pwd, ls, and the full absolute path.
Cron job does not run	Check crontab -l and systemctl status cron; use absolute paths.
Log file not created	Create the log folder with mkdir -p ~/logs and check the Cron command.
Backup folder is empty	Check that ~/Documents exists and that the script ran successfully.

Important theory points

• **Job scheduling:** Running a command or script automatically at a specified time or interval.
• **Cron:** A time-based Linux scheduler for recurring jobs.
• **Bash script:** A file containing shell commands executed by Bash.
• **Systemd timer:** A timer unit that triggers a related systemd service.
• **Logging:** Saving program output and errors in a file for later inspection.
• **Job dependency:** A task that must wait for another task or condition before it runs.

Possible viva questions

1. What is Cron? A Linux utility used to schedule recurring tasks automatically.
2. What does chmod +x do? It gives a file execution permission.
3. What is the difference between crontab -e and crontab -l? The first edits the schedule; the second displays it.
4. What does */5 mean in the minute field? Every five minutes.
5. What does >> do? It appends output to a file.
6. What does 2>&1 do? It redirects standard error to the same destination as standard output.
7. Why use absolute paths in Cron? Cron may run with a limited environment and a different working directory.
8. What is the difference between Cron and systemd timers? Cron uses crontab schedules; systemd timers use timer and service units.

Practical sequence to memorize

```
mkdir -p ~/scripts ~/logs
cd ~/scripts
nano backup.sh
# Type the Bash script and save
chmod +x backup.sh
./backup.sh
ls ~/backup
crontab -e
# Add the Cron schedule, then Ctrl+O, Enter, Ctrl+X
crontab -l
cat ~/logs/backup.log
```

Exam tip: Remember the main sequence: Create folders → Write script → Give permission → Run manually → Schedule with Cron → Verify with crontab -l → Check logs.